

1

Modelos generativos y su ecosistema

Qué es un modelo generativo, qué tipos existen, para qué se usan y cómo se integran en una aplicación mediante APIs, con el resumidor de tickets de Nube Sur como caso.

APRENDIZAJE ESPERADO 1 · TEXTUAL DEL PLAN FORMATIVO

ANALIZAR EL ROL DE LOS MODELOS GENERATIVOS EN EL DESARROLLO DE APLICACIONES, DESCRIBIENDO SUS TIPOS, CASOS DE USO, TECNOLOGÍAS ASOCIADAS Y REPRESENTANDO ARQUITECTURAS FUNCIONALES QUE INTEGREN ESTOS MODELOS MEDIANTE APIS.

PLAN FORMATIVO

PF1822 · Especialización en Desarrollo con IA

MÓDULO

MA04576 · INTRODUCCIÓN A MODELOS GENERATIVOS Y CONSUMO POR API · 21 h

CASO DE ESTUDIO

Nube Sur, empresa ficticia

TIEMPO DE LECTURA

20 minutos, más la autocomprobación

Qué vas a lograr

Al terminar esta lectura tendrás la base para demostrar los criterios de evaluación del aprendizaje esperado 1, textuales del plan formativo:

- 1.1** CLASIFICA LOS TIPOS DE MODELOS GENERATIVOS IDENTIFICANDO SUS CARACTERÍSTICAS Y APLICACIONES MÁS FRECUENTES EN LA ORGANIZACIÓN.

- 1.2** REPRESENTA UN DIAGRAMA FUNCIONAL DE UNA APLICACIÓN CON IA GENERATIVA, EXPLICANDO LA FUNCIÓN DE CADA COMPONENTE DENTRO DE LA ARQUITECTURA.

- 1.3** ANALIZA LA ELECCIÓN DE LA ARQUITECTURA EN FUNCIÓN DE CRITERIOS COMO ESCALABILIDAD, SEGURIDAD O MANTENIMIENTO.

Cómo está organizada

- | | |
|--|---|
| <ul style="list-style-type: none"> 1 IA generativa frente a IA tradicional <hr/> 2 Tipos por lo que generan <hr/> 3 Familias técnicas <hr/> 4 Casos de uso <hr/> 5 Arquitecturas <hr/> | <ul style="list-style-type: none"> 6 Componentes <hr/> 7 Tecnologías y bibliotecas <hr/> 8 Elegir una arquitectura <hr/> + En síntesis, práctica, autocomprobación y glosario <hr/> |
|--|---|

Contenidos del plan y dónde se tratan

Contenido del plan formativo (textual)	Dónde
1. MODELOS GENERATIVOS Y ECOSISTEMA TECNOLÓGICO	Antes de empezar
CONCEPTOS GENERALES DE IA GENERATIVA: QUÉ ES LA IA GENERATIVA Y PARA QUÉ SIRVE.	Sección 1
DIFERENCIAS ENTRE IA GENERATIVA Y IA TRADICIONAL BASADA EN REGLAS.	Sección 1
TIPOS DE IA GENERATIVA: TEXTO, IMAGEN, AUDIO Y VIDEO.	Sección 2
TIPOS DE MODELOS: AUTORREGRESIVOS, DE DIFUSIÓN Y BASADOS EN TRANSFORMERS.	Sección 3
CASOS DE USO: GENERACIÓN DE TEXTO, GENERACIÓN DE CÓDIGO, SÍNTESIS DE IMÁGENES, QA, SUMMARIZATION Y GENERACIÓN DE TESTS.	Sección 4
ECOSISTEMA TECNOLÓGICO DE LA IA.	Sección 7
ARQUITECTURAS: REQUEST-RESPONSE Y PIPELINES.	Secciones 5 y 8
COMPONENTES: CLIENTE, SERVIDOR API, MODELO DE IA, MÓDULO DE PREPROCESAMIENTO Y BASE DE DATOS/VECTOR STORE.	Sección 6
FLUJO DE DATOS END-TO-END: TECNOLOGÍAS Y LIBRERÍAS: OPENAI API, HUGGING FACE TRANSFORMERS, TENSORFLOW, PYTORCH, LANGCHAIN Y DIFFUSERS.	Secciones 6 y 7

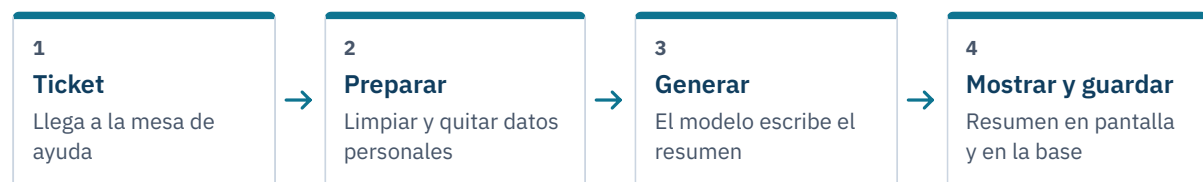
Antes de empezar

CONTENIDO DEL PLAN 1. MODELOS GENERATIVOS Y ECOSISTEMA TECNOLÓGICO

Los modelos generativos pasaron en pocos años de la investigación al trabajo diario de los equipos de desarrollo. Hoy una aplicación puede resumir textos, responder preguntas o generar código llamando a un modelo por API, sin entrenar nada. Pero integrar un modelo no es solo hacer una llamada: hay que saber qué tipo de modelo resuelve la tarea, dónde encaja en la arquitectura y qué decisiones de seguridad, costo y mantenimiento implica.

El caso del módulo es **Nube Sur**, una empresa ficticia de software cuya mesa de ayuda recibe decenas de tickets al día, muchos con HTML, firmas y datos personales mezclados con el problema real. El equipo quiere un **resumidor de tickets**: una aplicación que muestre, junto a cada ticket, una oración que diga qué falla y dónde. En esta lectura vas a entender qué hay detrás de esa aplicación antes de escribir una línea de código.

EL RESUMIDOR DE TICKETS, EN GRANDE



1 IA generativa frente a IA tradicional

CONTENIDO DEL PLAN CONCEPTOS GENERALES DE IA GENERATIVA: QUÉ ES LA IA GENERATIVA Y PARA QUÉ SIRVE. · DIFERENCIAS ENTRE IA GENERATIVA Y IA TRADICIONAL BASADA EN REGLAS.

La **inteligencia artificial generativa** es la rama de la IA que **produce contenido nuevo**: texto, código, imágenes, audio o video. Un modelo generativo aprende, a partir de grandes cantidades de datos, los patrones de ese contenido, y con ellos genera salidas que no estaban escritas en ninguna parte. Sirve para redactar, resumir, traducir, responder preguntas, programar o crear imágenes a partir de una descripción.

La **IA tradicional basada en reglas** funciona distinto: decide con condiciones que escribió una persona. "Si el ticket contiene la palabra *factura*, asignarlo al área de finanzas" es una regla. Estos sistemas son predecibles, fáciles de auditar y baratos, pero solo resuelven los casos que alguien anticipó. No pueden escribir un resumen, porque no hay regla que describa todos los resúmenes posibles.

	IA basada en reglas	IA generativa
Cómo decide o produce	Condiciones escritas por personas	Patrones aprendidos de datos
Qué entrega	Una decisión entre opciones fijas	Contenido nuevo

	IA basada en reglas	IA generativa
Predecible	Sí: la misma entrada da la misma salida	No del todo: puede variar entre corridas
Casos no previstos	No los resuelve	Los aborda, con riesgo de error
Explicable	Se lee la regla	Difícil de explicar paso a paso

EJEMPLO · NUBE SUR · Las dos en la mesa de ayuda

Nube Sur usa **reglas** para asignar cada ticket a un área según palabras clave, porque es rápido, gratis y auditable. Usa un **modelo generativo** para resumir, porque cada ticket está escrito de manera distinta. Las dos conviven: la IA generativa no reemplaza a las reglas donde las reglas bastan.

ERROR FRECUENTE · Usar un modelo donde bastaba una regla

Calcular si un ticket venció su plazo de respuesta es aritmética: no necesita un modelo generativo. Usarlo agrega costo, demora y la posibilidad de un error que una regla no comete. Reserva la IA generativa para lo que requiere producir lenguaje.

2 Tipos por lo que generan

CONTENIDO DEL PLAN TIPOS DE IA GENERATIVA: TEXTO, IMAGEN, AUDIO Y VIDEO.

La forma más directa de clasificar la IA generativa es por el tipo de contenido que produce. Cada tipo tiene sus modelos, sus usos frecuentes en una organización y sus propios cuidados.

Tipo	Qué genera	Usos frecuentes en una organización
Texto	Respuestas, resúmenes, traducciones, código	Atención al cliente, documentación, apoyo a la programación
Imagen	Imágenes a partir de una descripción o de otra imagen	Material de marketing, prototipos visuales
Audio	Voz a partir de texto, música, efectos	Locución de videos, asistentes de voz
Video	Clips a partir de texto o de imágenes	Videos de capacitación, piezas publicitarias

Muchos modelos actuales son **multimodales**: reciben o generan más de un tipo de contenido, por ejemplo una imagen y una pregunta sobre ella. Para un equipo de desarrollo, el tipo de salida define qué modelo buscar, qué API usar y cómo validar el resultado.

EJEMPLO · NUBE SUR · Qué necesita Nube Sur

El resumidor necesita **texto a texto**: entra un ticket, sale una oración. Si en el futuro el equipo quisiera leer tickets que llegan con una captura de pantalla del error, necesitaría un modelo multimodal que acepte imagen y texto. La tarea define el tipo de modelo, no al revés.

3 Familias técnicas

CONTENIDO DEL PLAN TIPOS DE MODELOS: AUTORREGRESIVOS, DE DIFUSIÓN Y BASADOS EN TRANSFORMERS.

Por dentro, los modelos generativos se agrupan en familias según cómo generan. Los **modelos autorregresivos** producen la salida un elemento a la vez: predicen el siguiente token a partir de todos los anteriores, lo agregan y repiten. Así funcionan los modelos de lenguaje que escriben texto y código. Los **modelos de difusión** parten de ruido aleatorio y lo refinan en muchos pasos hasta llegar a una imagen o un audio que corresponde a la descripción.

Los **transformers** son una arquitectura de red neuronal, basada en un mecanismo llamado atención, que permite relacionar cada parte de la entrada con todas las demás. Hay transformers **codificadores**, que convierten un texto en una representación numérica útil para buscar o clasificar; **decodificadores**, que generan texto de forma autorregresiva; y **codificador–decodificador**, que leen una entrada completa y generan una salida transformada, como una traducción.

IDEA CLAVE

"Autorregresivo" y "de difusión" describen **cómo se genera**; "transformer" describe la **arquitectura**. Por eso se superponen: la mayoría de los modelos de lenguaje actuales son transformers autorregresivos.

Familia	Cómo genera	Salida típica	Ejemplos de tecnologías
Autorregresivos	Predicen el siguiente token, uno a la vez	Texto, código	Modelos de lenguaje tipo GPT, Llama o Mistral
De difusión	Refinan ruido en muchos pasos	Imágenes, audio	Stable Diffusion, biblioteca Diffusers
Transformers codificadores	Convierten texto en vectores; no generan texto libre	Embeddings, clasificación	Familia BERT, modelos de embeddings
Transformers codificador–decodificador	Leen la entrada completa y generan una salida transformada	Texto a texto, audio a texto	Familias T5 y BART, Whisper

EJEMPLO · NUBE SUR · Dos familias en el mismo proyecto

Para resumir, Nube Sur usa un **transformer autorregresivo**: genera la oración token a token. Para encontrar tickets parecidos a uno nuevo, usa un **transformer codificador** que convierte cada ticket en un vector numérico (un embedding) y compara vectores. Son dos modelos distintos para dos tareas distintas.

4 Casos de uso

CONTENIDO DEL PLAN CASOS DE USO: GENERACIÓN DE TEXTO, GENERACIÓN DE CÓDIGO, SÍNTESIS DE IMÁGENES, QA, SUMMARIZATION Y GENERACIÓN DE TESTS.

Los casos de uso de la IA generativa en el desarrollo de aplicaciones se repiten en todas las industrias. Cada uno trae un beneficio y un riesgo que el equipo debe controlar.

Caso de uso	Ejemplo en Nube Sur	Cuidado principal
Generación de texto	Borrador de respuesta al cliente	Revisar tono y datos antes de enviar
Generación de código	Función para limpiar tickets	Probar el código; puede no funcionar
Síntesis de imágenes	Ilustraciones para el centro de ayuda	Derechos de uso de las imágenes
QA (preguntas y respuestas)	Responder dudas con la documentación del producto	Que responda solo con la documentación, sin inventar
Summarization (resumen)	Una oración por ticket	Que no agregue datos que no están en el ticket
Generación de tests	Casos de prueba para una función	Revisar que las pruebas verifiquen lo correcto

EJEMPLO · NUBE SUR · Generar pruebas a partir de una función

El equipo le entrega al modelo la función `limpiar(texto)` y le pide cinco pruebas con `pytest`, incluidos casos con HTML y firmas. El modelo propone las pruebas en segundos; el equipo las ejecuta, descarta una que verificaba algo equivocado y conserva las otras cuatro. El modelo acelera, pero la persona valida.

ERROR FRECUENTE · Tomar la salida como verdad

Un modelo generativo produce texto probable, no texto verificado. Un resumen puede incluir una causa que el ticket no menciona, y un código generado puede tener un error sutil. Cada caso de uso necesita su forma de validar: revisión humana, pruebas automáticas o comparación con una referencia.

5 Arquitecturas

CONTENIDO DEL PLAN ARQUITECTURAS: REQUEST-RESPONSE Y PIPELINES.

Hay dos formas básicas de integrar un modelo en una aplicación. En la arquitectura de **solicitud-respuesta** (*request-response*), la aplicación hace una llamada, el modelo responde y la aplicación muestra el resultado. Es simple, pero la persona espera mientras el modelo genera, y cada solicitud es independiente.

En una arquitectura de **pipeline**, el procesamiento se organiza en **etapas encadenadas**, donde la salida de una es la entrada de la siguiente: limpiar, generar, evaluar, guardar. Cada etapa se prueba y se mejora por separado, y el pipeline puede procesar muchos elementos en lote, incluso fuera de horario, sin que nadie espere en pantalla.

UN PIPELINE DE TICKETS



EJEMPLO · NUBE SUR · Las dos en el resumidor

Cuando una persona abre un ticket y pide "resumir ahora", la aplicación usa **solicitud-respuesta**: una llamada, un resumen en pocos segundos. Cada noche, un **pipeline** procesa en lote todos los tickets del día que aún no tienen resumen, los evalúa y los guarda. Muchas aplicaciones combinan las dos arquitecturas.

6 Componentes

CONTENIDO DEL PLAN COMPONENTES: CLIENTE, SERVIDOR API, MODELO DE IA, MÓDULO DE PREPROCESAMIENTO Y BASE DE DATOS/VECTOR STORE. · FLUJO DE DATOS END-TO-END: TECNOLOGÍAS Y LIBRERÍAS: OPENAI API, HUGGING FACE TRANSFORMERS, TENSORFLOW, PYTORCH, LANGCHAIN Y DIFFUSERS.

Una aplicación con IA generativa tiene cinco componentes típicos:

Componente	Función en el resumidor de Nube Sur
Cliente	La aplicación de la mesa de ayuda en el navegador: muestra el ticket y su resumen, y nunca conoce la clave
Servidor API	El servidor interno que recibe el ticket, orquesta el flujo y es el único que guarda la clave del modelo
Módulo de preprocesamiento	Limpia el texto y quita datos personales antes de enviarlo: menos tokens y menos riesgo
Modelo de IA	El modelo externo que genera el resumen y los embeddings, al que se accede por API

Componente	Función en el resumidor de Nube Sur
Base de datos o vector store	Guarda tickets, resúmenes y embeddings; el <i>vector store</i> permite buscar tickets parecidos por significado

El **flujo de datos de extremo a extremo** (*end-to-end*) recorre esos componentes en orden. Seguirlo con un ticket concreto es la mejor forma de entender la arquitectura, y de encontrar dónde puede fallar.

EJEMPLO · NUBE SUR · El recorrido del ticket T1

Marta escribe desde Contabilidad que no puede iniciar sesión tras cambiar su contraseña. Su ticket llega con HTML y una firma con su correo. El **cliente** lo envía al **servidor API**. El servidor lo pasa al **módulo de preprocesamiento**, que quita etiquetas, firma y correo. El texto limpio va al **modelo** en una solicitud POST y vuelve el resumen: "El usuario no puede iniciar sesión en la app de facturación tras cambiar su contraseña". El servidor guarda ticket, resumen y embedding en la **base de datos** y devuelve el resumen al cliente.

7 Tecnologías y bibliotecas

CONTENIDO DEL PLAN ECOSISTEMA TECNOLÓGICO DE LA IA · FLUJO DE DATOS END-TO-END: TECNOLOGÍAS Y LIBRERÍAS: OPENAI API, HUGGING FACE TRANSFORMERS, TENSORFLOW, PYTORCH, LANGCHAIN Y DIFFUSERS.

El **ecosistema tecnológico de la IA** es el conjunto de servicios y bibliotecas que un equipo combina para construir. Cada pieza resuelve una parte distinta del flujo de datos:

Tecnología	Qué resuelve	Cuándo la usarías en Nube Sur
API de OpenAI	Acceso por HTTP a modelos alojados por la empresa: generación, embeddings y más	El resumidor, sin administrar servidores de modelos
Hugging Face Transformers	Biblioteca de Python para descargar y ejecutar miles de modelos abiertos	Probar un modelo abierto de resumen en la propia máquina
PyTorch y TensorFlow	Frameworks de aprendizaje profundo sobre los que se entrenan y ejecutan los modelos	Ajustar un modelo con tickets propios
LangChain	Framework para orquestar aplicaciones con modelos de lenguaje: prompts, cadenas de pasos, recuperación de documentos y agentes	Un asistente que consulta la documentación antes de responder
Diffusers	Biblioteca de Hugging Face para modelos de difusión	Generar ilustraciones para el centro de ayuda

En este módulo trabajarás con la API de OpenAI y la de Hugging Face, porque es la forma más directa de integrar un modelo. Las demás bibliotecas reaparecen en los módulos siguientes del plan: la orquestación con frameworks, la recuperación aumentada por generación con almacenamiento vectorial y el ajuste fino de modelos.

EJEMPLO · NUBE SUR · Qué biblioteca en qué componente

En el resumidor, el **servidor API** usa la API de OpenAI a través de la clase `ClienteIA` que construirás en el aprendizaje esperado 2. Si el equipo decidiera ejecutar un modelo abierto en sus servidores, el componente **modelo de IA** pasaría a usar Hugging Face Transformers sobre PyTorch, y el resto de la arquitectura no cambiaría.

8 Elegir una arquitectura

CONTENIDO DEL PLAN ARQUITECTURAS: REQUEST-RESPONSE Y PIPELINES.

No hay una arquitectura correcta en abstracto: se elige con criterios. Cuatro son los que más pesan en una aplicación con IA generativa:

- **Escalabilidad.** ¿Aguanta diez veces más tickets? Un servidor que encola solicitudes y un pipeline que procesa en lote escalan mejor que llamadas directas desde cada pantalla.
- **Seguridad.** ¿Dónde están la clave de la API y los datos personales? La clave vive solo en el servidor, en una variable de entorno, y los datos personales se quitan antes de enviar el texto a un servicio externo.
- **Costo.** ¿Cuántos tokens consume cada ticket? Limpiar el texto antes de enviarlo reduce tokens, y procesar en lote permite controlar el gasto.
- **Mantenimiento.** ¿Se puede cambiar de modelo o de proveedor sin reescribir todo? Encapsular las llamadas en un solo módulo lo hace posible.

EJEMPLO · NUBE SUR · La justificación del resumidor

El equipo eligió un servidor API intermedio con preprocesamiento por **seguridad**, porque la clave y los datos personales quedan del lado del servidor y nunca llegan al navegador. También por **escalabilidad**, porque el servidor puede encolar tickets y procesarlos en lote sin cambiar la aplicación de la mesa de ayuda.

ERROR FRECUENTE · La clave en el cliente

Una aplicación que llama al modelo directamente desde el navegador tiene que llevar la clave consigo, y todo lo que llega al navegador puede leerlo cualquiera. Una clave expuesta se usa para gastar a cuenta de la empresa. La clave va siempre en el servidor.

En síntesis

- La IA generativa produce contenido nuevo aprendido de datos; la IA basada en reglas decide con condiciones escritas por personas, y las dos pueden convivir.
- Por lo que generan, hay modelos de texto, imagen, audio y video, y muchos son multimodales.
- Autorregresivo y de difusión describen cómo se genera; transformer describe la arquitectura.
- Los casos de uso más frecuentes son texto, código, imágenes, preguntas y respuestas, resúmenes y pruebas, y cada uno necesita su forma de validar.
- Solicitud–respuesta sirve para respuestas inmediatas; un pipeline encadena etapas y procesa en lote.
- Los componentes son cliente, servidor API, preprocesamiento, modelo y base de datos o vector store.
- La arquitectura se elige por escalabilidad, seguridad, costo y mantenimiento; la clave siempre queda en el servidor.

SIGUIENTE PASO

Para practicar

- **Actividad 1, "Un cliente de API para el resumidor", parte A.** Dibujas el diagrama funcional del resumidor, explicas cada componente y justificas la arquitectura con dos criterios.
- **Cuadro comparativo de familias de modelos.** Agregas en cada fila un caso de uso para Nube Sur.
- **Documentación oficial.** La de OpenAI (platform.openai.com/docs) y la de Hugging Face (huggingface.co/docs) describen los modelos disponibles y sus usos.

Autocomprobación

Responde con tus palabras antes de mirar las respuestas. Si dudas en alguna, vuelve a la sección que se indica.

- 1 ¿En qué se diferencia una IA basada en reglas de una IA generativa? Da un ejemplo de tarea de Nube Sur para cada una.

- 2 Describe el recorrido de un ticket por el resumidor, nombrando los cinco componentes en orden.

- 3 ¿Por qué la clave de la API se guarda en el servidor y no en la aplicación del navegador? ¿Con qué criterio de elección de arquitectura se relaciona?

RESPUESTAS

1. La basada en reglas decide con condiciones que escribió una persona, por ejemplo asignar un ticket a finanzas si contiene "factura". La generativa produce contenido nuevo aprendido de datos, por ejemplo el resumen de cada ticket (sección 1).
2. El cliente envía el ticket al servidor API; el servidor lo pasa al módulo de preprocesamiento, que lo limpia; el texto limpio va al modelo de IA, que devuelve el resumen; el servidor guarda todo en la base de datos o vector store y devuelve el resumen al cliente (sección 6).
3. Porque todo lo que llega al navegador puede leerlo cualquiera, y una clave expuesta permite gastar a cuenta de la empresa. Se relaciona con el criterio de seguridad (secciones 6 y 8).

Glosario

Embedding	Representación de un texto como vector de números, que permite comparar textos por su significado.
IA generativa	Rama de la inteligencia artificial que produce contenido nuevo, como texto, código, imágenes o audio.
Modelo autorregresivo	Modelo que genera la salida un token a la vez, prediciendo cada uno a partir de los anteriores.
Modelo de difusión	Modelo que genera imágenes o audio refinando ruido aleatorio en muchos pasos.
Pipeline	Arquitectura de etapas encadenadas, donde la salida de cada etapa es la entrada de la siguiente.
Solicitud–respuesta	Arquitectura en que la aplicación hace una llamada al modelo y espera su respuesta para mostrarla.
Transformer	Arquitectura de red neuronal basada en atención, en la que se apoyan la mayoría de los modelos de lenguaje actuales.
Vector store	Base de datos que guarda embeddings y permite buscar los más parecidos a uno dado.